1. Define the Problem Statement

As the System Architect, my responsibility in this stage is to critically evaluate and validate the technical feasibility of the identified problem, ensuring that the proposed solution can be realistically implemented within the constraints of time, resources, and team capability.

New students, visitors, and even some staff members often experience difficulty navigating the campus environment due to unfamiliarity with building locations, lecture halls, laboratories, and administrative offices. This challenge results in wasted time, missed lectures, reduced productivity, and frustration.

Currently, users rely on:

- Static printed maps (which are often outdated)
- Verbal directions (which may be unclear or inconsistent)
- Trial-and-error navigation

From a technical standpoint, I analysed whether a web-based information system can effectively address these challenges. I determined that a structured system can:

- Store campus locations in a searchable format
- Allow users to query locations using keywords
- Display structured information instantly
- Provide step-by-step navigation instructions

I further ensured that:

- The problem does not require advanced technologies such as GPS or AI
- The solution can run entirely within a web browser
- The system can operate offline if needed (using local JSON data)

2. Gather Requirements

In this stage, I defined detailed functional, non-functional, and technical requirements, ensuring that each requirement is measurable and directly supports the system's objectives.

2.1 Functional Requirements

The system must provide the following capabilities:

1. Search Functionality
    - Users must be able to search by:
        - Location name (e.g., “Library”)
        - Category (e.g., “Laboratory”, “Administration”)
    - The system must support partial matches (e.g., “lib” → “Library”)
    - Search results must display within 1–2 seconds
2. Display Location Details
    - Each result must include:
        - Location name
        - Category
        - Description (minimum 1–2 sentences)
        - Directions
    - Results must be displayed in a structured card format
3. Directions Feature
    - Provide step-by-step textual directions
    - Directions must include:
        - Starting point (e.g., Main Gate)
        - Landmarks (e.g., “next to Library”)
    - Must be clear and easy to follow
4. Campus Map Display
    - A static campus map image must be included
    - Map must be accessible from the main interface
    - Optional: clickable areas (if time permits)
5. Data Management
    - System must store at least 15–25 campus locations
    - Each location must follow a standardized structure

2.2 Non-Functional Requirements

- Performance:

    - Search results must load in ≤ 2 seconds
- Usability:
    - Interface must require no training
    - Navigation must be intuitive
- Reliability:
    - System must not crash during normal use
- Scalability:
    - Additional locations can be added without code changes
- Maintainability:
    - Code must be modular and well-commented

2.3 Technical Requirements

- HTML5 for semantic structure
- CSS3 for responsive layout and styling
- JavaScript (ES6) for logic implementation
- JSON file containing structured data:
    - Minimum 15 entries
    - Maximum 50 entries (for scalability testing)

3. Define Project Scope

As the architect, I ensured that the scope is precisely defined to avoid over-engineering while maintaining functionality.

In Scope

- Keyword-based search system
- Static campus map integration
- Structured data display
- Text-based navigation instructions
- Modular file organization

Out of Scope

- GPS tracking or live navigation
- Database systems (e.g., MySQL)
- Mobile app deployment (Android/iOS)
- Real-time updates or APIs

Scope Justification

This ensures:

- Completion within 21 days
- Reduced system complexity
- Focus on core functionality

4. Assess Feasibility

I conducted a multi-dimensional feasibility analysis:

Technical Feasibility

- Entire system runs in a browser → no server required
- JSON replaces database → reduces complexity
- JavaScript handles all logic

Operational Feasibility

- Users only need basic computer skills
- System accessible via any modern browser

Time Feasibility

- Total duration: 21 days (3 weeks)
- Tasks divided into manageable daily activities

Resource Feasibility

- Tools required:
    - Code editor (e.g., VS Code)
    - Web browser
    - GitHub

No additional cost or infrastructure required.

5. Identify Risks

Technical Risks

1. Data Inconsistency
    - Missing fields in JSON
    - Incorrect formatting
2. Integration Failure
    - UI not displaying fetched data correctly
3. Search Inefficiency
    - Slow or inaccurate results
4. Version Conflicts
    - Multiple team members editing same files

Mitigation Strategies

- Define a strict JSON schema
- Use Git version control with clear commit messages
- Conduct daily testing of features
- Assign file ownership to avoid conflicts

6. Choose the Tech Stack

Selected Technologies

- HTML → structure
- CSS → layout and styling
- JavaScript → functionality

- JSON → data storage
- GitHub → collaboration

Technical Justification

- Lightweight and fast
- No backend dependency
- Easy debugging and testing
- Suitable for academic project scope

7. System Architecture & Design

7.1 Detailed Architecture Explanation

The system follows a 3-layer architecture model, ensuring separation of concerns:

Presentation Layer

- Handles all user interactions
- Displays:
    - search bar
    - results
    - map
- Built using HTML & CSS

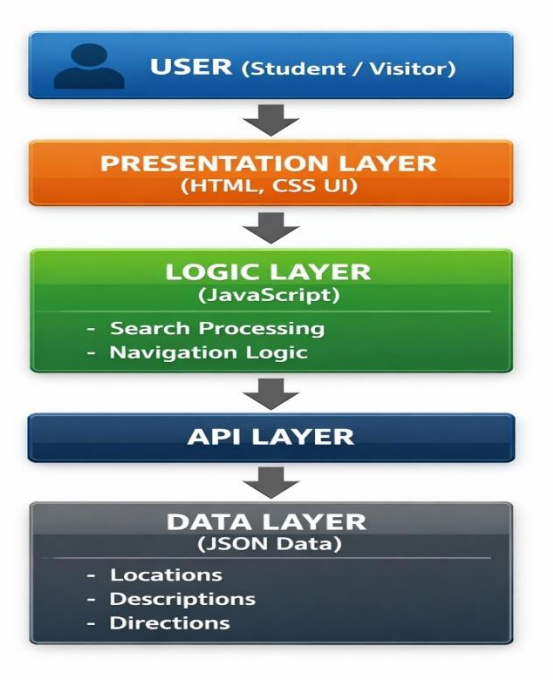
Logic Layer

- Handles:
    - search algorithm
    - filtering logic
    - data parsing
- Built using JavaScript

Data Layer

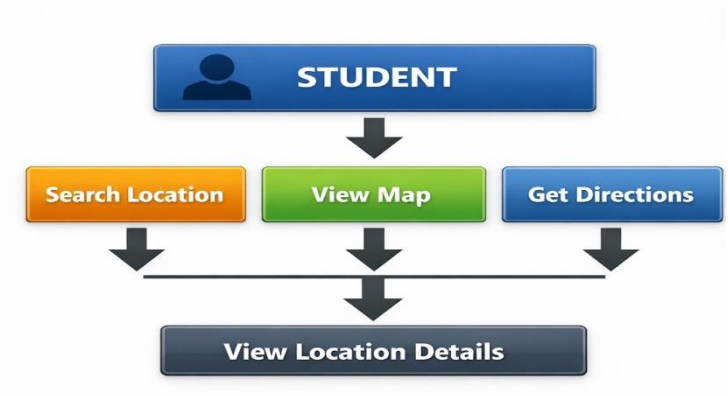
- Stores structured JSON data

- Contains:
    - 15–25 location records
- Easily extendable



Explanation:
This diagram illustrates how the user interacts with the system through the presentation layer, how the logic layer processes requests, and how data is retrieved from the data layer.

7.1.2 Use Case Diagram



Explanation:
This diagram shows the interactions between the user and the system.

7.2 Detailed System Flow

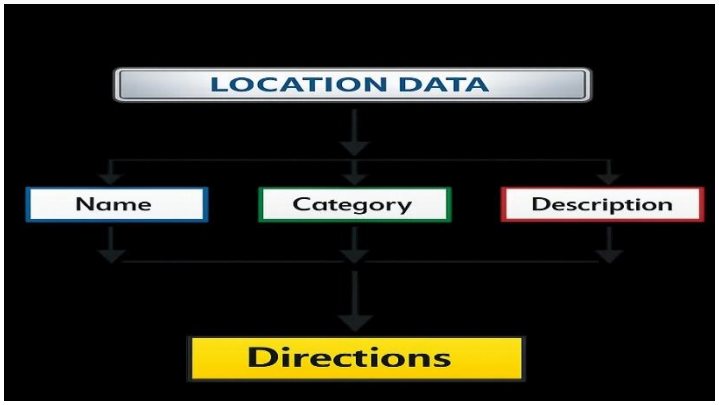
1. User enters search query
2. JavaScript captures input
3. System filters JSON data
4. Matching results are returned
5. Results displayed dynamically
6. User selects location → sees directions

Explanation:
This diagram represents how data flows through the system from user input to output.

7.3 Data Structure

```
{
 "id": 1,
 "name": "Library",
 "category": "Academic",
 "description": "Main study area with books and computers",
 "directions": "From the main gate, walk straight for 100m, turn left at Admin Block",
```

```
  "keywords": ["books", "study", "library"]
}
```



8. Assign Roles & Responsibilities

I ensured clear technical ownership and coordination:

- Manager → monitors deadlines and approvals
- Designer → creates UI layouts and styling
- Programmer → implements logic and features
- Architect → defines structure, validates integration, ensures consistency

9. Create the Project Timeline

Total Duration: 21 Days

Week 1: Planning & Architecture (Days 1-7)

- Day 1: Problem definition & requirement analysis
- Day 2: Finalize system architecture
- Day 3: Design JSON data structure
- Day 4: Create diagrams (architecture, flow, use case)
- Day 5: Setup GitHub repository
- Day 6: UI wireframes (Designer)
- Day 7: Review and finalize design

Week 2: Development (Days 8-14)

- Day 8–9: HTML structure implementation
- Day 10–11: CSS styling and layout
- Day 12: JavaScript search functionality
- Day 13: Data integration (JSON + UI)
- Day 14: Initial testing

Week 3: Testing & Finalization (Days 15-21)

- Day 15–16: Functional testing
- Day 17: Debugging
- Day 18: UI improvements
- Day 19: Final testing
- Day 20: Documentation & upload to GitHub
- Day 21: Final review and submission

10. Document the Plan

All architecture decisions are documented, including:

- System design
- Data format
- Workflow

11. Get Approval & Sign Off

I ensure stakeholders understand:

- System structure
- Technical decisions
- Implementation approach

Conclusion

As the System Architect, I ensure the system is:

- Technically sound
- Efficiently structured
- Completed within 21 days

I provide technical leadership and ensure successful integration of all components.